Wear Leveling with Flash Memory

Working with flash memory means being able to store memory in predefined memory sectors where it will remain stored even if the power supply is cut off. Flash memory has a defined lifespan. For example, the microcontroller I am using is the NXP s32k144, which has 512 KB of internal flash memory with a lifespan of around 10,000 to 100,000 erase/write cycles, according to the manufacturer.

Obviously, I can write to it as many times as I want and wait for the memory cells to die slowly (or quickly) and then change memory sectors when necessary. But what if I want to avoid this and increase its lifespan by a factor of 10 or 100? Is that possible? I am pleased to say that yes, it is possible.

The answer is to implement a Wear Leveling algorithm. What it does is distribute write operations almost evenly. It prevents wear from being concentrated in specific sectors. Example: Instead of always writing to the same sector, it rotates between different sectors/blocks.

You define the number of memory addresses with which each sector begins that you will use for a memory instance that, let's say, you will use to store the data obtained from a temperature sensor that occupies 8 bytes. You will need to categorize what each byte you are going to store is in order to have an order, for example: instance number, write counter, and the length of bytes sensed value. There you would have your frame:

Written in C language, the structure definition would look like this:

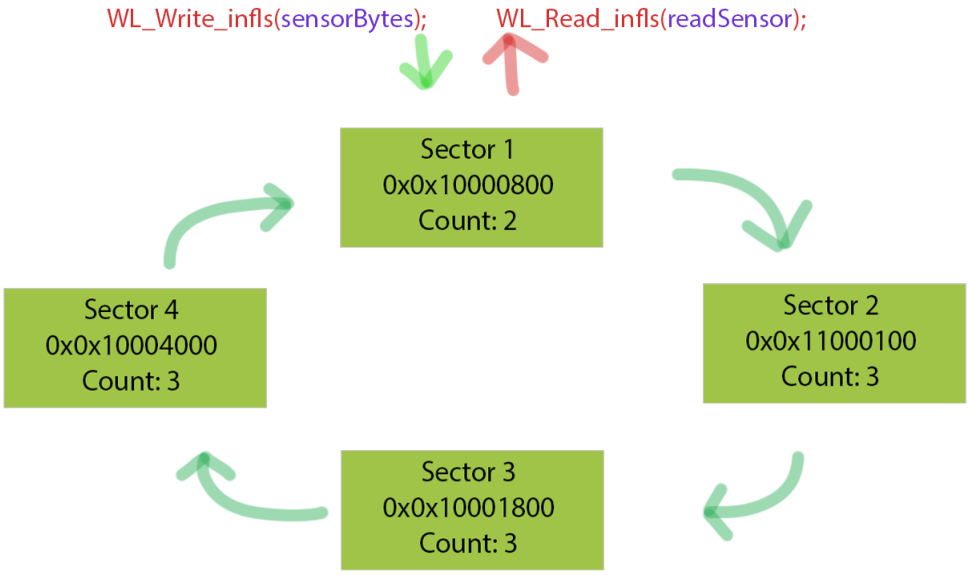
```
typedef struct
{
uint32_t dataId;      // Data identifier
uint32_t writeCount; // Deed counter
uint8_t  data[8];     // Maximum User Data
} FlashSector;
```

Where it would be seen as already stored in one of the “n” designated sectors

```
Memory 1 @ 10004000
10004000  00 00 00 00 38 00 00 00  A1 B2 C3 D4 E5 F6 07 17
10004020  FF FF FF FF FF FF FF FF  FF FF FF FF FF FF FF FF
```

Figure 1 Ozone Debugger

In the next write operation, choose the next sector to write to. Choose the sector with the fewest writes to write to there. To read, save the last location or memory address of the previously written sector to read in RAM.



The lifespan of your flash memory without wear leveling will depend critically on your writing patterns.

Example 1:

Event scenario every 5 seconds

Parameters:

- Maximum cycles: 10,000
- Sector size: 4KB
- Total sectors available: 128KB / 4KB = 32 sectors
- Writes/day**:** 17,280 (every 5 seconds)
- Data/write: 256 bytes
- Sectors/write: 1 (because 256B < 4KB)

Out wear leveling (always using 2 sectors):

= 10,000 / (17,280 / (8KB/256B))

= 10,000 / (17,280 / 32)

= 10,000 / 540

≈ 18.5 días

With wear leveling (using all 32 sectors):

= (10,000 × 32) / (17,280 / (4KB/256B))

= 320,000 / (17,280 / 16)

= 320,000 / 1,080

≈ 296 días (≈0.8 años)

Example 2:

Configuration scenario every hour

Parameters:

• Writes/day: 24

• Data/write: 1KB

• Sectors/write: 1 (1KB ≤ 4KB)

Without wear leveling:

= 10,000 / (24 / (4KB/1KB))

= 10,000 / 6

≈ 1,666 days ≈ 4.5 years

With basic wear leveling (32 sectors):

= (10,000 × 32) / (24 / (4KB/1KB))

= 320,000 / 6

≈ 53,333 días ≈ 146 years

Realistic value with practical factors (temperature, safety margin):

≈ **20-30 years**

Simplified Spreadsheet

Parameters	Without Wear Leveling	With Basic Wear Leveling	With Advanced Wear Leveling
Sectors Used	2	32	96*
Effective Cycles	10,000	320,000	960,000
Writes/day	540	1,080	108**
Lifespan (days)	18.5	296	8,888
Lifespan (years)	0.05	0.8	24.3

*96 = 32 sectors × 3 (redundancy)

**Grouping 10 writes into one

Results vary because they depend on the compression rate of the data used, the efficiency of the algorithm used, the percentage of flash dedicated to wear leveling, and the environmental conditions in the environment.

Correction Factors Applied

To make the calculations more realistic, I considered:

1. **Safety factor 3x**: To allow for margin
2. **Partial use of flash:** Only 10-30% for dynamic data
3. **Temperature degradation:** Reduces useful life in automotive and other environments
4. **Non-perfect distribution:** Actual wear leveling is not 100% efficient

Partial use of flash for wear levelingl: This is a good idea for dynamic data. Let me explain; Instead of using all the flash memory capacity available for dynamic storage, you can reserve a portion for frequently changing data, while the rest is used for:

- Firmware code (which is rarely updated)
- Semi-static data (settings that change occasionally)
- Spare blocks

It is a good idea to allocate between 10% and 30% of the flash memory. For example, for a microcontroller with 128KB and 4KB as the sector size, we would have up to 32 sectors available (128KB/4KB = 32 sectors) where we allocate 10% = 3 sectors for dynamic data. And now with 30% = 9-10 sectors, this allows for sufficient rotation without wasting resources. If only 10 sectors receive wear instead of 32:

$$\text{Lifespan} \approx \text{Maximum cycles} \times (10 / 32) \approx 3.2\text{x improvement}$$

Critical firmware sectors will not suffer unnecessary wear.

Example:

In an event data recorder (EDR) system:

- 30% dynamic flash (38.4KB):
  - Stores acceleration data (sampled at 100Hz)
  - 10-minute circular buffer
- 70% static flash (89.7KB):
  - Main firmware
  - Calibration settings
  - Critical error logs

3x safety factor: The 3x safety factor is a critical design margin that may be a good idea to apply in embedded systems to ensure long-term reliability. Where we have three times more resources than theoretically necessary to compensate for:

1. Variations in hardware specifications

2. Adverse operating conditions

3. Non-uniform degradation

Imperfect distribution: When I say that wear leveling does not have perfect distribution, I mean that in practice it is impossible to ensure that all memory cells wear out exactly uniformly. These are the key factors:

1. Practical limitations:

a) Data fragmentation:

// Example: Data of different sizes prevent ideal rotation

```
typedef struct {
    uint32_t    id;             // 4 bytes
    uint8_t     sensor_data;    // 1 byte (95% of the sector unused)
} SmallData;
```

b) Access Patterns

Some data is updated much more frequently than others.

Example:

- Hot data: Counters, timestamps (frequent writing)
- Cold data: Configurations (infrequent writing)

2. Technical Causes

a) Metadata Overhead: When we implement control information that each sector carries, this also wears down the sector.

```
typedef struct {
    uint32_t    write_count;  // Updated in EVERY deed → Hot spot
    uint32_t    data_id;
    uint8_t     data[4088];   // 4KB sector - 8B metadata
} FlashSector;
```

b) Hardware limitations

- Flash requires erasure by complete blocks (4KB in S32K144)
- An individual byte cannot be written without rewriting the entire sector

3. Consequences in Calculations

Adjusted Formula:

Actual service life = Theoretical service life × Efficiency factor

Where:

Typical efficiency factor: 0.7 to 0.9 (70-90%)

In automotive applications, we use 0.5 (50%) to be conservative.

If you want to see some code written, here is my repo:

Bitbucket: lab_micros / simple-wearlevelingx — Bitbucket